
Padrões Criacionais

Factory Method & Builder



Prof. Dr. João Choma

UEM

github.com/JoaoChoma

Fundamentos



A sequência segue a lógica:

problema → metáfora → padrão → Java

- 1 Por que padrões de projeto existem?
- 2 Por que Factory Method e Builder são criacionais?
- 3 Factory Method: qual objeto concreto criar?
- 4 Builder: como construir um objeto complexo?
- 5 Aplicação em exemplos Java testáveis.





Eles resolvem **problemas recorrentes de projeto**, não servem apenas para “embelezar” o código.



Sem padrão

A equipe reinventa soluções e espalha decisões por várias classes.

Com padrão

A intenção fica explícita e surge um vocabulário compartilhado.



Categoria	Pergunta	Exemplos
Criacionais	Como criar objetos?	Factory Method, Builder, Singleton
Estruturais	Como compor objetos?	Adapter, Decorator, Facade
Comportamentais	Como objetos interagem?	Strategy, Observer, Command

Factory Method e Builder retiram do cliente parte da responsabilidade de construir objetos.

Factory Method



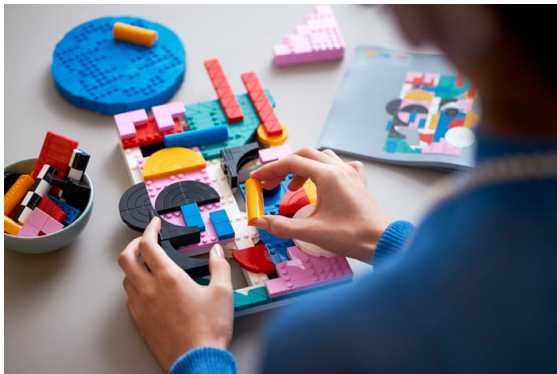
A fábrica de brinquedos

O cliente pede um tipo de brinquedo; a estrutura de criação decide qual objeto entregar.

- “Quero um carrinho” → Carrinho
- “Quero um robô” → Robo
- “Quero um avião” → Aviao

Ideia central

Quem pede não precisa conhecer os detalhes de construção.

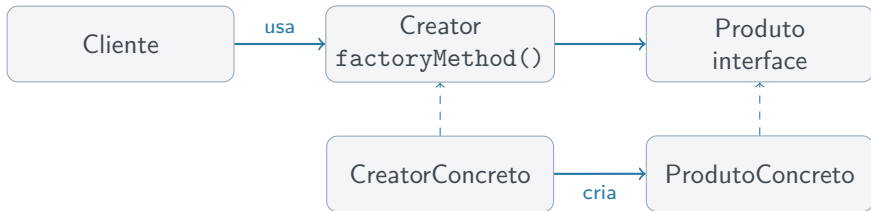




Resolver a pergunta: qual classe concreta deve ser instanciada?

Problema	Solução pretendida
Vários new espalhados	Encapsular a criação
Cliente conhece classes concretas	Cliente depender de abstração
Novo tipo altera muitos pontos	Isolar a decisão de criação
Testes ficam rígidos	Permitir substituições

É criacional porque controla a criação de produtos concretos.



- O Creator declara o método fábrica.
- O Creator concreto redefine o método e escolhe o produto.
- O cliente utiliza a abstração, não a implementação concreta.



```
class NotificacaoService {  
    void enviar(String tipo, String destino, String msg) {  
        if (tipo.equals("EMAIL")) {  
            new EmailNotificacao().enviar(destino, msg);  
        } else if (tipo.equals("SMS")) {  
            new SmsNotificacao().enviar(destino, msg);  
        } else if (tipo.equals("PUSH")) {  
            new PushNotificacao().enviar(destino, msg);  
        }  
    }  
}
```

- O serviço conhece todos os tipos concretos.
- Criação e regra de envio estão misturadas.
- Adicionar uma opção exige modificar o serviço.



```
interface Notificacao {  
    void enviar(String destino, String mensagem);  
}  
  
enum TipoNotificacao { EMAIL, SMS, PUSH }  
  
class NotificacaoFactory {  
    static Notificacao criar(TipoNotificacao tipo) {  
        return switch (tipo) {  
            case EMAIL -> new EmailNotificacao();  
            case SMS   -> new SmsNotificacao();  
            case PUSH  -> new PushNotificacao();  
        };  
    }  
}
```

Centraliza a criação, mas ainda concentra a escolha em um switch.



Simple Factory	Factory Method GoF
Classe central decide por <code>if</code> ou <code>switch</code>	Subclasses redefinem o método de criação
Não é um dos 23 padrões GoF	É um padrão GoF
Centraliza a criação	Delega a criação por polimorfismo

Definição

Factory Method define uma interface de criação e permite que subclasses decidam qual classe instanciar.



```
abstract class NotificacaoCreator {
    protected abstract Notificacao criarNotificacao();

    public void enviar(String destino, String mensagem) {
        Notificacao n = criarNotificacao();
        n.enviar(destino, mensagem);
    }
}

class EmailCreator extends NotificacaoCreator {
    @Override
    protected Notificacao criarNotificacao() {
        return new EmailNotificacao();
    }
}

class SmsCreator extends NotificacaoCreator {
    @Override
    protected Notificacao criarNotificacao() {
        return new SmsNotificacao();
    }
}
```

Builder



O objeto final pode ser complexo, mas sua construção acontece em etapas:

- 1 escolher o pão;
- 2 adicionar queijo e vegetais;
- 3 escolher o molho;
- 4 finalizar o pedido.

Ideia central

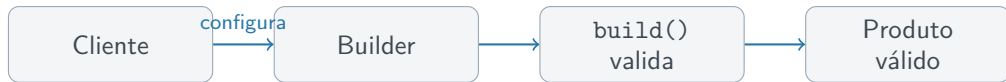
Substituir um construtor gigante por passos nomeados e encadeáveis.





Resolver a pergunta: **como construir um objeto complexo sem construtores confusos?**

Problema	Solução com Builder
Muitos parâmetros	Construção passo a passo
Ordem difícil de memorizar	Métodos com nomes explícitos
Muitas opções e sobrecargas	Configurações encadeáveis
Objeto pode nascer inválido	<code>build()</code> valida antes de criar



```
Relatorio relatorio = new Relatorio.Builder("Vendas", "Joao")
    .formato("PDF")
    .confidencial(true)
    .build();
```



```
Relatorio r = new Relatorio(  
    "Vendas Q4",           // titulo  
    "Joao",                // autor  
    LocalDate.of(2026, 10, 1), // inicio  
    LocalDate.of(2026, 12, 31), // fim  
    "PDF",  
    true,  
    false,  
    "CONFIDENCIAL"  
);
```

- O que significam true e false?
- E se as datas forem invertidas?
- Cada nova opção aumenta o construtor.



```
Relatorio r = new Relatorio.Builder("Vendas Q4", "Joao")
    .periodo(
        LocalDate.of(2026, 10, 1),
        LocalDate.of(2026, 12, 31))
    .formato("PDF")
    .incluirGraficos(true)
    .confidencial(true)
    .build();
```

Leitura

Cada método explica a configuração. A ordem dos parâmetros opcionais deixa de ser um problema.



```
public class Relatorio {
    private Relatorio(Builder b) { /* copia valores */ }

    public static class Builder {
        public Builder(String titulo, String autor) { /* ... */ }

        public Builder formato(String formato) {
            this.formato = formato;
            return this;
        }

        public Builder confidencial(boolean valor) {
            this.confidencial = valor;
            return this;
        }

        public Relatorio build() {
            validar();
            return new Relatorio(this);
        }
    }
}
```



O objeto final só deve existir depois que suas regras forem satisfeitas.

- Verificar campos obrigatórios.
- Garantir que a data inicial não seja posterior à final.
- Aplicar valores padrão aos campos opcionais.
- Impedir combinações incompatíveis.

Invariante

Se a validação falhar, o Builder lança uma exceção e o objeto inválido não é criado.

Comparação e decisão



Critério	Factory Method	Builder
Pergunta	Qual objeto criar?	Como montar o objeto?
Problema	Múltiplas implementações	Objeto complexo
Foco	Escolha da classe concreta	Processo de construção
Exemplo	Creator escolhe e-mail ou SMS	Relatorio.Builder
Resultado	Polimorfismo de criação	Objeto legível e válido

Regra rápida

Factory Method escolhe entre variações; Builder organiza uma construção longa ou cheia de opções.



Factory Method

Encapsula a escolha da implementação concreta e entrega uma abstração.

Builder

Encapsula o passo a passo e produz um objeto válido.

Em comum

Reduzem acoplamento e retiram criação do código cliente.

Ambos dão nome a uma solução recorrente para a criação de objetos.



Factory Method use quando o problema é escolher uma implementação sem acoplar o cliente a ela.

Builder use quando o objeto possui muitos parâmetros, opções ou validações.

Simple Factory uma decisão central por switch; útil, mas diferente do Factory Method GoF.

Perguntas para decidir

- O problema é **qual implementação instanciar**? Considere Factory Method.
- O problema é **como montar um objeto complexo**? Considere Builder.

Referência: Gamma et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.